			set of criteria can produce zero or more
GeneratedPassword	**A password created according to selected criteria**	**value,strengthRating**	**Each GeneratedPassword is produced from one PasswordCriteria**

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Conceptual classes only: things and records from the problem domain, with attributes and associations. No operations, no data types, no database tables, no screen classes.

Every item the actor column of section 4.2 mentions is an attribute somewhere in this model. Every association has a multiplicity at both ends.

Budget: one page including the diagram.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

Lot (name, zone, capacity) 1 ── 0..* Space (number, type) ; Space 1 ── 0..* OccupancyReading (time, state) ; Permit (type, valid until) * ── 1..* Lot "is valid in" ; Building (name) * ── * Lot "is near" (walking time).

5.2 Object interaction models

Copy the block below for at least two of the use cases in 4.1; preferably including your riskiest.

5.2.1 UC-01 Generate New Password

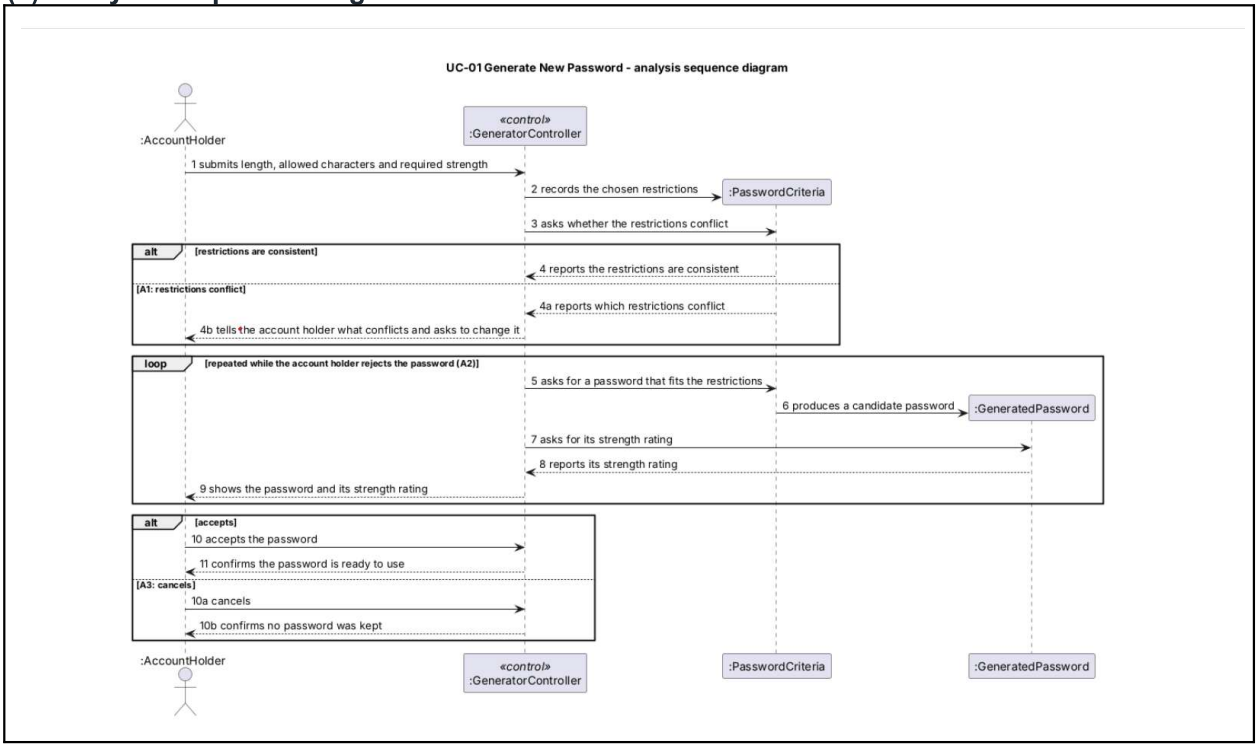
(a) Non-trivial steps. Step (4) is non-trivial: The system stores the restrictions, verifies that they are not contradictory (A1), generates a password and evaluates its length. Steps (1), (3) and (5) are actor actions. Only options and/or input is shown in steps (0) and (2), and step (6) only confirms the accepted password.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	**AccountHolder**	**Submits the password restriction**	**Length, allowedCharacters, requiredStrength**	**GeneratorController**
2	**GeneratorController**	**Records the chosen restrictions**	**Length, allowedCharacters, requiredStregnth**	**PasswordCriteria**
3	**GeneratorController**	**Ask whether the restrictions conflict**		**PasswordCriteria**
4	**PasswordCriteria**	**Report the restrictions are consistent**		**GeneratorController**

4a	PasswordCriteria	Reports which restrictions conflict	Length, requiredStrength	GeneratorController
4b	GeneratorController	Tells the account holder what conflicts and ask to change it	Conflicting restrictions	AccountHolder
5	GeneratorController	Ask for a password that fits restrictions		PasswordCriteria
6	PasswordCriteria	Produces a candidate passwords	value	GeneratedPassword
7	GeneratorController	Ask for its strength rating	value	GeneratedPassword
8	GeneratedPassword	Reports its strength rating	StrengthRating	GeneratorController
9	GeneratorController	Shows the password and its strength rating	Value, StrengthRating	AccountHolder
10	AccountHolder	Accepts the password	value	GeneratorController
10a	AccountHolder	cancel		GeneratorController
10b	GeneratorController	Confirms no password was kept		AccountHolder
11	GeneratorController	Confirms the password is ready to use	value	AccountHolder

(c) Analysis sequence diagram.



5.2.2 UC-02 Store New Credential

(a) Non-trivial steps. Step (4) is not trivial: It verifies that the details are complete, verifies that the vault is unlocked, verifies for any duplication and creates the credential and stores it in the vault. Steps (2) and (0) only request input or only display the existing list, respectively, while steps (1) and (3) are actor actions.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	AccountHolder	Submit's the new credential details	Service, username, password	CredentialController
2	CredentialController	Checks that every detail is present	Service, username, password	CredentialController
2a	CredentialController	Names the missing detail and keeps what was entered	Missing detail	AccountHolder
3	CredentialController	Ask whether the vault is unlocked	LockStatus	Vault
4	Vault	Reports the the vault is unlocked	LockStatus	CredentialController
4a	Vault	Reports the vault is locked	LockStatus	CredentialController
4b	CredentialController	Ask the account holder to unlock before saving		AccountHolder
5	CredentialController	Ask whether a credential exist for this service and username	Service, username	Vault
6	Vault	Reports no matching credential		CredentialController
6a	Vault	Reports a matching credential	Existing credential	CredentialController
6b	CredentialController	Shows the existing credential and ask to replace or cancel	Existing credential	AccountHolder
7	CredentialController	Creates the credential from the entered details	Service, username, password	Credential
8	CredentialController	Ask the vault to add the credential	Credential	Vault
9	Vault	Confirms the credential is stored		CredentialController
9a	Vault	Reports the credential is not		CredentialController

		stored		
9b	**CredentialController**	**Tells the account holder nothing was saved and to reenter**		**AccountHolder**
10	**CredentialController**	**Confirms the save and show the credential in the list**	**Service, username**	**AccountHolder**

(c) Analysis sequence diagram.

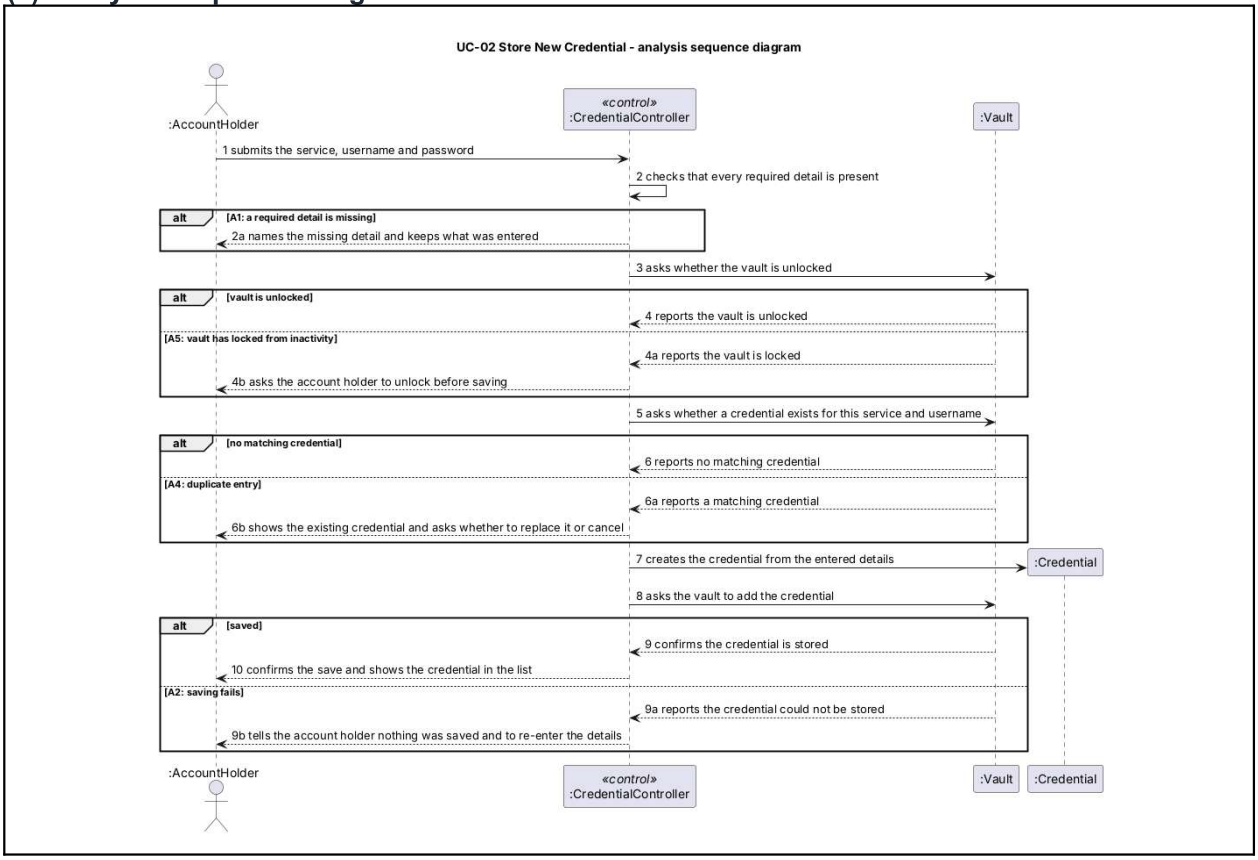


Figure 5.2.x Analysis sequence diagram for UC-xx; message numbers match the scenario table

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Write each step as a declarative sentence first, subject, verb, object, then fill the columns. Tables filled directly produce rows with no subject.

One message per table row, same numbers. Use the four translation cases from Lesson 14; the diagram is a conversion of the table, not a new drawing.

Messages are verb phrases, not operation signatures. "asks for the routes calling at the stop" is analysis; "getRoutes(stopId: int): List<Route>" is Chapter 9 and is not credited here.

Every object on the diagram is a class in 5.1 or an interaction object you name and explain in one line. One object is clearly in control of the interaction.

Budget: one to one and a half pages per use case.